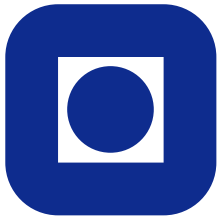


NTNU
Norwegian University of
Science and Technology

Binary exploitation and reverse engineering

Donn Morrison
Department of Computer Science

Outline

Native binaries

Reverse engineering

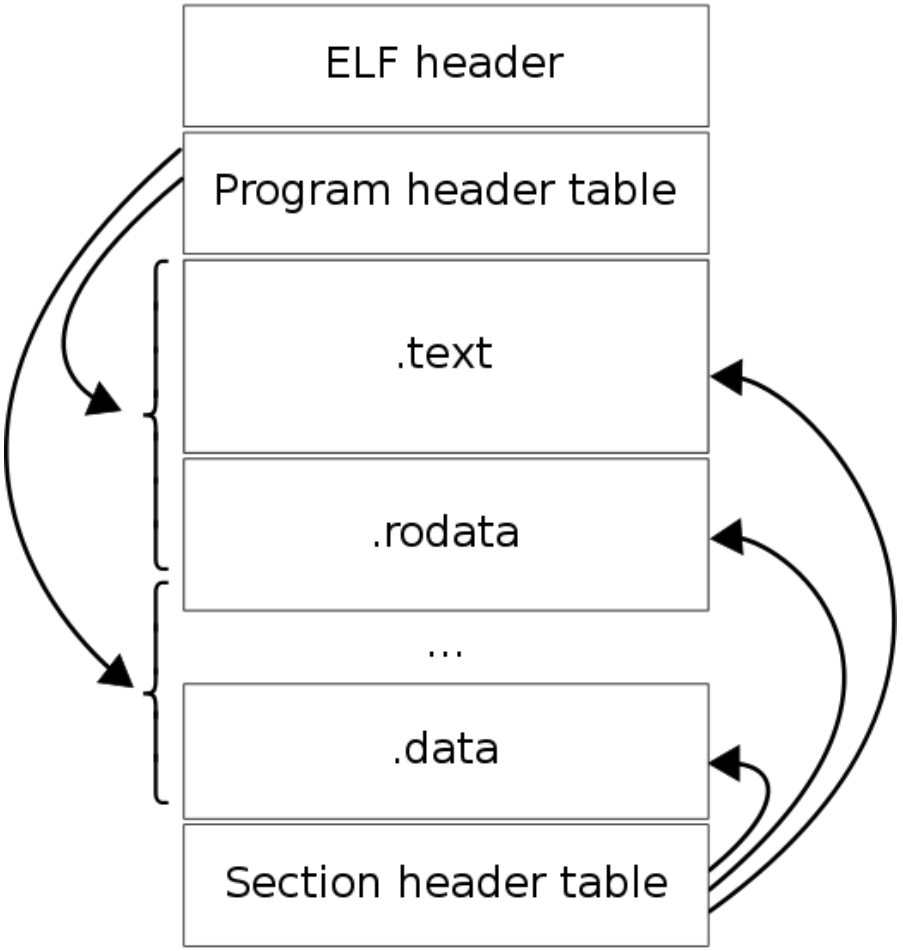
Tools

Introduction to binary exploitation

- C and C++ programs (memory management is hard)
- Used for systems programming, so use is widespread
- IoT devices, operating systems, etc...
- Rust aims to protect the programmer from a lot of common pitfalls

We focus on memory corruption issues and unsafe libc function calls.

ELF file format



- Executable and linkable format
- Standard executable format for Unix-like OSes
- Information in ELF can reveal some general security protections
  - Relocation read-only (RELRO)
  - Stack canary
  - NX stack
  - Position independent executable (PIE)
  - FORTIFY_SOURCE

https://upload.wikimedia.org/wikipedia/commons/e/e4/ELF_Executable_and_Linkable_Format_diagram_by_Ange_Albertini.png
https://developers.redhat.com/articles/2023/07/04/developers-guide-secure-coding-fortifysource

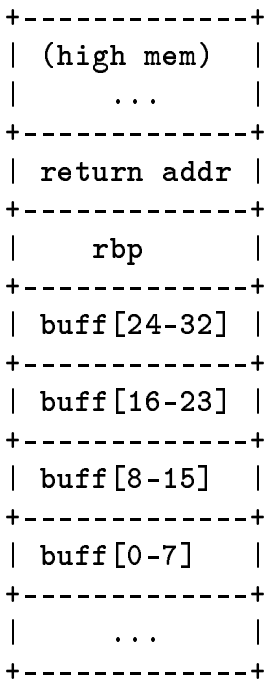
Buffer overflow

- Stack-based
- Heap-based

```
// bo.c buffer overflow example
int main()
{
    char buff[32];
    gets(buff);
    puts(buff);
}
```

```
$ gcc -o /tmp/bo /tmp/bo.c -no-pie
$ python -c "print('A'*40)" | /tmp/bo
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Segmentation fault (core dumped)
```

Stack (64-bit)



Buffer overflow cont.

```
$ checksec /tmp/bo
[*] '/tmp/bo'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE (0x400000)
```

Mitigations

- Address space layout randomisation (ASLR)
- Stack canary detects stack smashing (Read me! `https://www.sans.org/blog/stack-canaries-gingerly-sidestepping-the-cage/`)
- Non-executable (NX) stack/heap
- Position independent executable (PIE)

A word about shellcode

```
8049000:       31 c9                   xor    %ecx,%ecx
8049002:       6a 0b                   push   $0xb
8049004:       58                      pop    %eax
8049005:       51                      push   %ecx
8049006:       68 2f 2f 73 68          push   $0x68732f2f  // "//sh"
804900b:       68 2f 62 69 6e          push   $0x6e69622f  // "/bin"
8049010:       89 e3                   mov    %esp,%ebx
8049012:       cd 80                   int    $0x80
```

Becomes:

```
char [] shellcode = "\x31\xc9\x6a\x0b\x58\x51\x68\x2f\x2f\x73\x68\x68\x2f\x62\x69\x6e\x89\xe3\xcd\x80";
```

- Shellcode is user input supplied in the context of an exploit
- Specially crafted instructions to execute code, typically a shell
- Non-executable stack/heap greatly reduces ability to run shellcode
- More on how we get around this later...

`https://www.exploit-db.com/shellcodes/46809`

Use-after-free/dangling pointer

```c
// Fedora 20 x64: gcc ./uaf.c
struct unicorn_counter { int num; };

int main() {
    struct unicorn_counter* p_unicorn_counter;
    int* run_calc = malloc(sizeof(int));
    *run_calc = 0;
    free(run_calc);                                          // <-- The free
    p_unicorn_counter = malloc(sizeof(struct unicorn_counter));
    p_unicorn_counter->num = 42;
    if (*run_calc) execl("/bin/gnome-calculator", 0);        // <-- The use
}
```

Use-after-free/dangling pointer cont.

Also possible on the stack if a pointer points to a stack location that goes out of scope

```
int main()
{
        int *pointer = NULL;

        if (1) {
                int var = 42;
                pointer = &var; // pointer gets stack address of var
        } // var falls out of scope

        pointer; // <-- dangling
}
```

Format string vulnerability

- User input passed directly to dangerous functions
- Family of *printf string formatting functions
- Ability to read and write from memory
- `https://en.wikipedia.org/wiki/Uncontrolled_format_string`

```c
// fs.c format string example
#include <stdio.h>
int main()
{
    char buff[32];
    fgets(buff,32,stdin);
    printf(buff);
}
```

Try it out, pass `%10$p` as `stdin`, and we get the next 10

Format string vulnerability cont.

From `man 3 printf`:

```
p      The void * pointer argument is printed in hexadecimal  (as  if  by
       %#x or %#lx).

n      The number of characters written so far is stored into the integer
       pointed to by the corresponding argument.  That argument shall  be
       an  int *, or variant whose size matches the (optionally) supplied
       integer length modifier.  No argument is converted.  (This  speci-
       fier  is  not supported by the bionic C library.)  The behavior is
       undefined if the conversion specification includes  any  flags,  a
       field width, or a precision.
```

https://stackoverflow.com/questions/7459630/how-can-a-format-string-vulnerability-be-exploited#7459758

Many, many more possible memory bugs

See `https://googleprojectzero.blogspot.com/2015/06/what-is-good-memory-corruption.html` for some good examples with popping calculators.

Return-oriented programming

- Non-executable stack made things difficult
- We can inject our own shellcode but cannot run it
- But...we can jump to arbitrary locations in .text (program code) of the running program, or in any shared libraries loaded into process memory
- We can search for *gadgets* of instructions ending in the `ret` instruction

```
$ gcc -o rop rop.c -no-pie
$ ROPgadget --binary rop
...
0x0000000000001013 : add esp, 8 ; ret
0x0000000000001012 : add rsp, 8 ; ret
0x00000000000011e1 : call qword ptr [r15 + rbx*8]
0x00000000000011e2 : call qword ptr [rdi + rbx*8]
0x0000000000001010 : call rax
0x00000000000010f7 : cmc ; add byte ptr cs:[rax], al ; test rax, rax ; je 0x1111 ; jmp rax
0x00000000000011e4 : fisttp word ptr [rax - 0x7d] ; ret
0x000000000000100e : je 0x1014 ; call rax
...
```

Return-oriented programming cont.

- Build up some “shellcode” using ROP chains
- Each time *ret* called, we pop the top of the stack off into the instruction pointer
- CPU continues executing at that location
- So our buffer overflow should overwrite the stack with a chain of the addresses of gadgets we want to execute in order

More in ROP:

- LiveOverflow ROP differently explaining part 1
  `https://www.youtube.com/watch?v=8Dcj19KGKWM`
- LiveOverflow Return-oriented programming tutorial
  `https://www.youtube.com/watch?v=zaQVNM3or7k`

Outline

Native binaries

Reverse engineering

Tools

Introduction

The process of understanding how a piece of software works (usually at a very low level)

- Reverse engineering (RE) contexts
  - Recreating outdated/incompatible software or protocols (no available source code)
  - Software freedom
  - Understanding malicious code
  - Exploiting vulnerabilities
  - Circumvent copy protection, DRM
  - Curiousity
  - Research (information security)

Examples

- Non-IBM PC-BIOS (launched the IBM-PC compatible industry)
- Samba `https://www.samba.org/` (Windows file share)
- Wine (Windows API)
- OpenOffice/LibreOffice (MS doc formats)
- ReactOS (binary compatability with Windows NT)

`https://en.wikipedia.org/wiki/Reverse_engineering#Reverse_engineering_of_software`

Techniques

- Analysis
  - Observing information exchange
  - Bus analysers, packet sniffers
  - JTAG for embedded systems
  - Low-level debuggers (SoftICE for debugging MS Windows)
- Binary disassembly
  - Study of machine code
  - Can be slow and tedious
- Decompilation
  - Use a tool to try to recreate the original source code (varying degrees of success)

Tools

- Disassembler (objdump, radare2, Ghidra, IDA) - static analysis
- Debugger (gdb (with peda), OllyDbg, ...) - dynamic analysis
- Hex, resource editors
- Firmware unpackers, e.g., binwalk

Disassembler - objdump

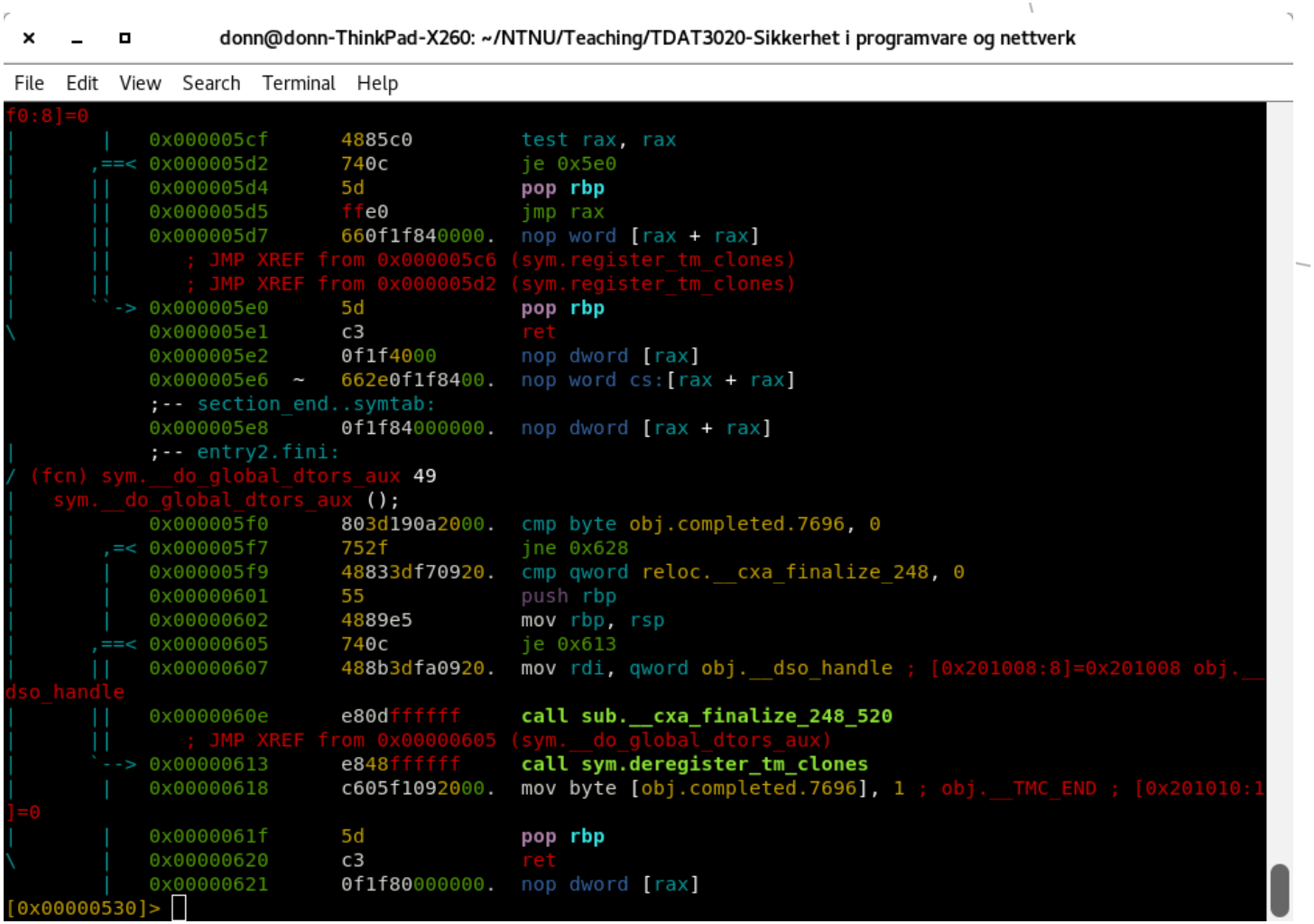
display information from object files

```
$ objdump -d /tmp/fork/a.out -M intel

/tmp/fork/a.out:     file format elf64-x86-64


Disassembly of section .init:
...
0000000000000530 <_start>:
 530:        31 ed                       xor    ebp,ebp
 532:        49 89 d1                    mov    r9,rdx
 535:        5e                          pop    rsi
 536:        48 89 e2                    mov    rdx,rsp
 539:        48 83 e4 f0                 and    rsp,0xfffffffffffffff0
 53d:        50                          push   rax
 53e:        54                          push   rsp
 53f:        4c 8d 05 7a 01 00 00        lea    r8,[rip+0x17a]        # 6c0 <__libc_csu_fini>
 546:        48 8d 0d 03 01 00 00        lea    rcx,[rip+0x103]        # 650 <__libc_csu_init>
 54d:        48 8d 3d e6 00 00 00        lea    rdi,[rip+0xe6]        # 63a <main>
 554:        ff 15 86 0a 20 00           call   QWORD PTR [rip+0x200a86]        # 200fe0 <__libc_start_ma
 55a:        f4                          hlt
 55b:        0f 1f 44 00 00              nop    DWORD PTR [rax+rax*1+0x0]
...
000000000000063a <main>:
 63a:        55                          push   rbp
 63b:        48 89 e5                    mov    rbp,rsp
 63e:        48 83 ec 10                 sub    rsp,0x10
 642:        e8 c9 fe ff ff              call   510 <fork@plt>
```

Disassembler - radare2

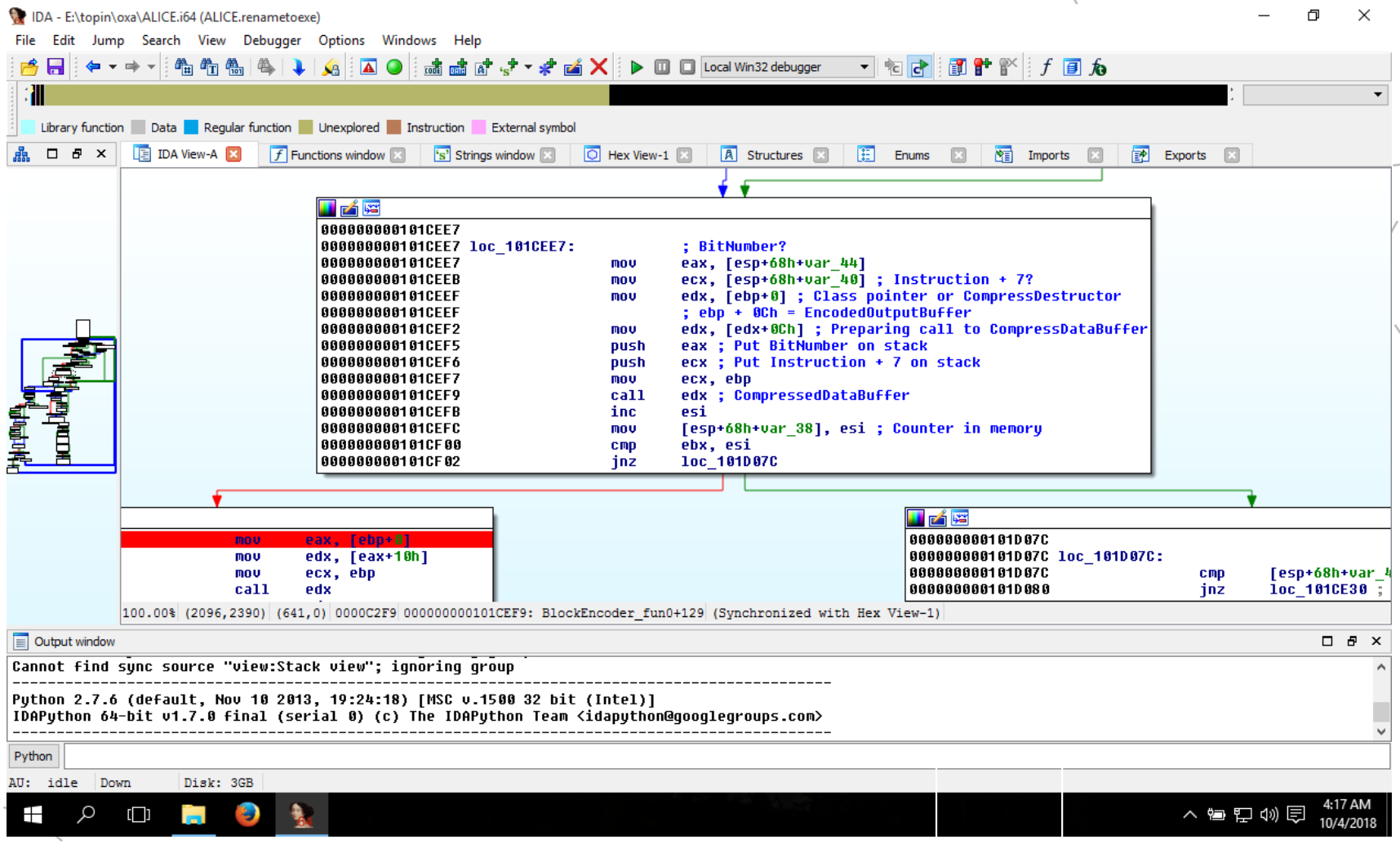
Advanced commandline hexadecimal editor, disassembler and debugger

Disassembler - radare2



Disassembler - IDA Pro

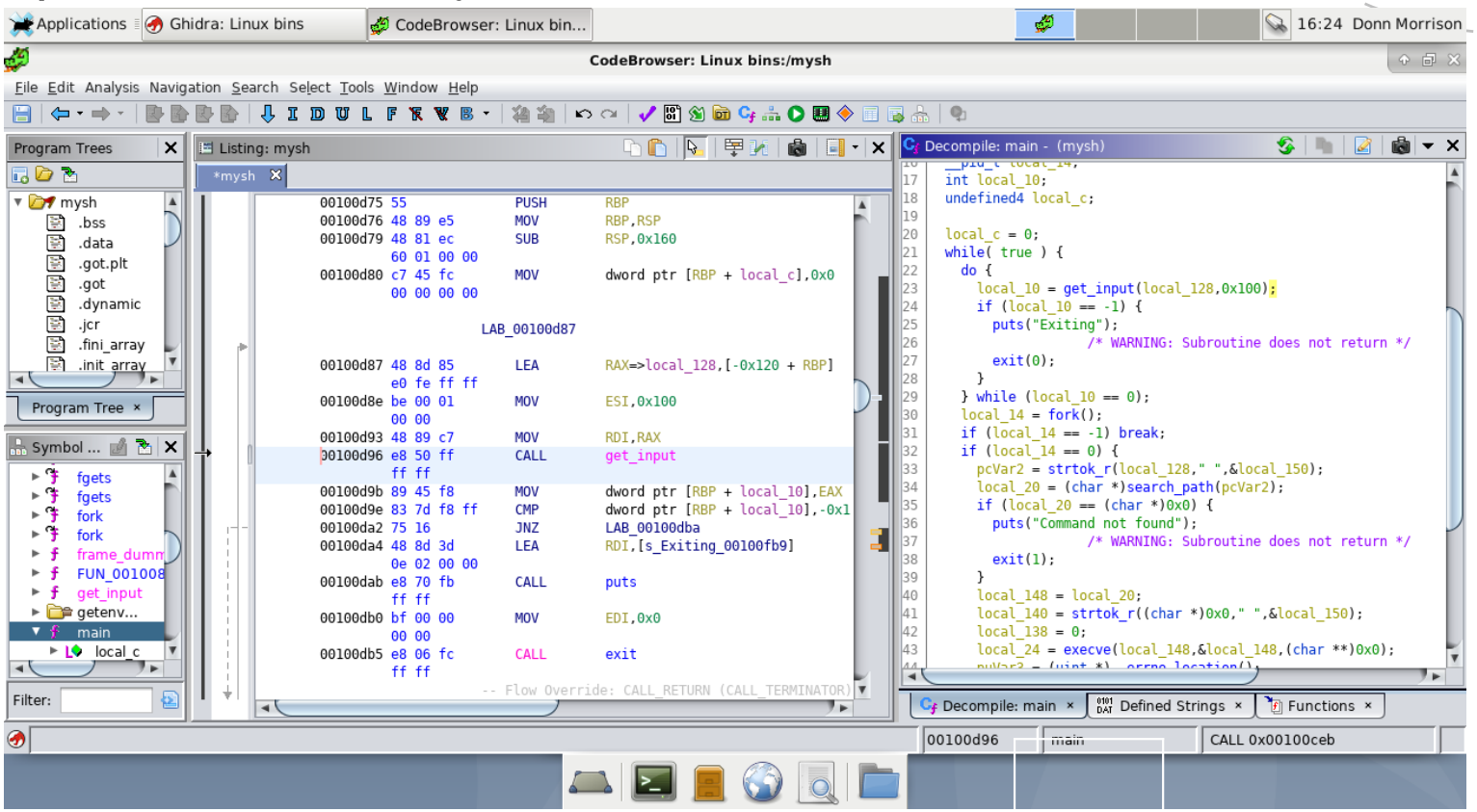
Multiarchitecture disassembler, debugger, decompiler



https://www.hex-rays.com/products/ida/index.shtml

Disassembler/decompiler - Ghidra

- Multiarchitecture disassembler, decompiler
- Developed by NSA's Research Directorate
- Open-souced early 2019



https://ghidra-sre.org/

Demos

- GNU debugger `gdb` with `peda`
  - Basic ROP exploit example - rop.c - ROPgadget/one_gadget/radare2 /R
- Reverse engineering an IoT binary
  - binwalk
  - strings
  - Ghidra

Pwning a router Pwn2own 2020

This short series documents recon, reverse engineering, and exploit writing to get a remote root shell on a router.

- How We Hacked a TP-Link Router and Took Home $55,000 in Pwn2Own `https://www.youtube.com/watch?v=zjafMP7EgEA` (20 mins)
- DNS Remote Code Execution: Finding the Vulnerability (Part 1) `https://www.youtube.com/watch?v=xWoQ-E8n4B0` (30 mins)
- DNS Remote Code Execution: Writing the Exploit (Part 2) `https://www.youtube.com/watch?v=YC0oc1U7kPA` (40 mins)

References

— C and C++ vulnerabilities
https://handouts.secappdev.org/handouts/2012/Yves%20Younan/C%20and%20C++%20vulnerabilities.pdf
— What is a good memory corruption vulnerability?
https://googleprojectzero.blogspot.com/2015/06/what-is-good-memory-corruption.html
— Format string (Stackoverflow)
https://stackoverflow.com/questions/7459630/how-can-a-format-string-vulnerability-be-exploited
— A bug hunter's diary: A guided tour through the wilds of software security. Tobias Klein. 2011.
— Gray Hat Hacking 3rd ed. (Book) http://pages.cs.wisc.edu/~ace/media/gray-hat-hacking.pdf
— Intro to Computer Security (Course syllabus)
http://pages.cs.wisc.edu/~ace/cs642-spring-2016.html